

Engineering Without Coding: A Practitioner Study of Operator-Driven AI Development

MohamadAlmstafa Azeez
Independent Researcher, Baghdad, Iraq

Abstract

This paper presents a practitioner case study of Kaplen, a production B2B SaaS system built in 32 days at a total tooling cost of USD 500 by an operator with formal computer science training who wrote no code directly. Claude served as the sole implementation agent across 18 documented conversation sessions comprising 3,662 turns. Analysis of the complete interaction corpus against system documentation identifies four failure modes that appeared consistently across sessions: configuration blindness (the model cannot observe runtime environment state), fragmentation (loss of architectural coherence across sessions), accumulation without consolidation (unbounded growth of system complexity without pruning), and scope creep (output expansion beyond stated requirements). For each failure mode, the operator developed compensating behaviors rooted in software engineering judgment. The system delivered 34 documented modules at approximately 300x cost compression compared to informal traditional development estimates. This case demonstrates that, for this operator and this system, software engineering competence and coding ability functioned as separable skills: architectural judgment was exercised entirely through prompting and structural decisions while implementation was delegated to the model. These findings are scoped to a single case and a single operator with engineering training; whether the same outcome is achievable with different operator profiles, domains, or systems is an open question this study does not address.

Keywords: AI-assisted development, operator-model interaction, software engineering, practitioner study, LLM, vibe coding, configuration blindness, fragmentation

1. INTRODUCTION

The rapid adoption of large language models as coding assistants has produced a growing body of empirical research. Most of this work measures one configuration: experienced software developers using AI tools to augment existing coding practice in established codebases [1, 2, 3]. A different configuration has received less systematic attention: a non-programmer operator with formal engineering training using an LLM as the sole implementation agent to build a system from scratch.

This configuration sits at the boundary of two active research threads. The first is vibe coding [4, 5, 6] — the practice of steering LLM-generated code through natural language prompts rather than writing code directly. The second is operator-model interaction [7], which examines the human role in directing AI systems toward structured outcomes. Neither thread has produced a longitudinal case study of a complete production system built by an engineering-trained operator with no coding involvement.

This paper fills that gap. Kaplen is a B2B SaaS platform built to serve Iraqi educational content creators, comprising curriculum ingestion, AI-assisted script generation, YouTube packaging, authentication, payment processing, analytics, and deployment infrastructure. The system was built in 32 calendar days across 18 documented conversation sessions at a tooling cost of USD 500 by an operator with a computer science background who wrote no code. All implementation was delegated to Claude through standard conversational interface.

The paper addresses three research questions:

RQ1: What failure modes appear consistently when an LLM is used as a sole implementation agent for a production system?

RQ2: What compensating behaviors does an engineering-trained operator develop in response to these failure modes?

RQ3: In this case, did software engineering skill and coding ability function as separable competencies?

Three contributions follow. First, a documented corpus of 18 operator-model conversation sessions cross-referenced against system documentation, covering a complete production development cycle. Second, four failure modes identified and defined with specific evidence across multiple sessions, independently validated by a rule-based classifier across 2,218 exchanges. Third, a precise reformulation of the popular claim that anyone can build SaaS with AI, distinguishing coding ability from engineering judgment as empirically separable in this case.

2. BACKGROUND AND RELATED WORK

2.1 Productivity Research on AI-Assisted Coding

Existing empirical work on AI coding assistants focuses on speed and quality metrics for professional developers. METR measured a 19% productivity loss with the Cursor AI coding assistant among experienced developers in established codebases [3]. GitClear analyzed 211 million lines of code and found that AI assistance reduced the refactoring fraction of commits [2]. Becker et al. [3] measured similar patterns in open-source developer productivity. A systematic review by Daniotti et al. [9] found that early-career developers showed no significant productivity benefit from AI tools, while experienced developers showed mixed results.

Critically, none of these studies measure the configuration examined here: an operator without coding ability directing a greenfield build. The populations studied — experienced developers augmenting existing practice — differ structurally from the present case.

2.2 Vibe Coding

The term vibe coding, coined by Andrej Karpathy in February 2025, describes a development approach where users steer LLM-generated code through natural language prompts

rather than writing code directly [4]. Ge et al. [5] provide the first systematic survey of vibe coding, drawing from over 1,000 research papers and identifying infrastructure components, failure patterns, and operator skill requirements. Chou et al. [6] conducted a grounded theory study of 20 vibe coding sessions (16 hours of live-streamed development, 254 prompts) and found that practitioners develop distinct prompting workflows and debugging approaches that differ from traditional development. Sarkar and Drosos [13] analyzed 8 hours of curated vibe coding sessions and found that experienced programmers use AI-generated code through rapid scanning and pattern matching rather than line-by-line review.

The present study differs from existing vibe coding research in two ways. First, the operator has no coding background — they cannot fall back to manual implementation when the model fails. Second, the scope is a complete production system rather than isolated tasks or sessions. These differences make the failure modes and compensating behaviors documented here distinct from those observed in vibe coding studies of experienced developers.

2.3 Failure Modes in LLM-Assisted Development

Cemri et al. [10] identify 14 failure modes in multi-agent LLM systems, organized into three categories: planning failures, execution failures, and response failures. Their taxonomy is the closest existing work to the failure modes identified here, though it was developed for multi-agent systems rather than single-model operator-directed builds. The context erosion problem — which Cemri et al. describe as gradual drift in alignment with original intent over multi-turn interactions — maps closely to the fragmentation failure mode documented in this study.

Codified context infrastructure [11] has emerged as a response to cross-session coherence failures in LLM-assisted development. AGENTS.md, introduced in July 2025 as an open standard for providing persistent context to AI coding agents, addresses the same problem that the operator in this study addressed through manual context pre-loading. The emergence of this infrastructure standard confirms that the fragmentation failure mode documented here is a recognized problem in the field, not an artifact of the specific system studied.

The "Should I Give Up Now?" study [12] examined LLM failure patterns in software engineering tasks and found that human oversight remains critical even for autonomous SE tasks, particularly because real-world programming problems are often ill-defined and lack clear solutions. This aligns with the operator compensating behaviors documented here, where the operator's primary contribution was maintaining problem definition and architectural constraints rather than implementation.

2.4 Human Oversight and Skill Requirements

A systematic review of LLM assistant impact on developer productivity [14] found that over-reliance and automation complacency are documented concerns among professional software engineers. Survey respondents report diminished ability to think independently after

sustained LLM use. The present study presents an inverse case: an operator whose engineering training enabled active oversight rather than passive acceptance of model output. Whether this oversight capacity is learnable without engineering training is an open question.

The operator role in human-AI systems has been formalized by Vulicevic [7] as a distinct layer between end-users and AI models. Sarkar [8] argues that intention specification — the ability to articulate architectural goals precisely — is the critical human skill in AI-assisted development. The operator behaviors documented in this study (context pre-loading, phase-gate prompting, clean rebuild decisions) represent concrete instances of intention specification under the conditions of a production build.

2.5 The Non-Programmer Can Build Narrative

Popular discourse holds that AI coding assistants democratize software development to the point where programming knowledge is unnecessary [4]. Ge et al. [5] note that this claim is widespread but empirically underexamined. Chou et al. [6] found in their vibe coding study that even experienced programmers struggle with complex architectural decisions in LLM-directed development. The present study provides a single data point relevant to this discussion: one engineering-trained operator successfully delivered a production system without coding. Whether this generalizes to operators without engineering training is not addressed by this study.

3. SYSTEM DESCRIPTION

3.1 Kaplen: Architecture and Scope

Kaplen is a multi-tenant B2B SaaS platform for Iraqi schoolteachers creating educational YouTube content from official curriculum materials. The system ingests curriculum PDFs, classifies content into subject and topic hierarchies, generates Arabic-language educational scripts using Claude, and packages output into YouTube-ready formats via a web dashboard. The live system is accessible at kaplen.app.

The final system comprises 34 documented modules across five categories: Core Generation Pipeline (essay_generator, long_form_generator, script_generator, study_tips, dedup), Data and Architecture (database, data_model, data_loader, synthesis), Infrastructure and Deployment (deployment, health, security, auth, api_endpoints), Analytics and Metrics (analytics, metrics, dashboard, batch), and Feature Extensions (callaway, validator, youtube_packager, payments, timeline).

3.2 Infrastructure

The system runs on AWS EC2 (t3.small) with PostgreSQL on RDS, S3 for curriculum storage, and Nginx as a reverse proxy. The Flask API serves a single-page web dashboard. JWT authentication and Stripe payments are integrated. The full stack was selected, specified, and debugged by the operator; all implementation was produced by Claude. No code was written directly by the operator at any point during the build.

The system processed 200 curriculum textbooks across 3 grade levels, 18 subjects, 108 topics, and 432 subtopics. The first video produced using a Kaplen-generated script, thumbnail, and title achieved 60% average view duration (AVD) and 7% click-through rate (CTR) on YouTube, above the platform average of 2-3% CTR and the 50% AVD threshold typically considered strong for educational content. Unit economics were validated at USD 67 per teacher, billed at the educational center level (each center comprising 7-20 teachers), producing a per-center price point of USD 469-1,340. One customer reached letter-of-intent stage before sales were paused for the research period. System uptime across the production period was 99.7%. These metrics establish that the system was production-grade and commercially viable. Sales were not pursued because the project was designed as a research instrument.

4. METHODOLOGY

4.1 Data Collection

Conversation logs were exported from Claude.ai in Markdown format across the complete development period. Eighteen conversation files were identified, comprising 3,662 total operator-model turns. Each file was parsed to extract operator prompts, model responses, file references, and tool actions. Conversations were cross-referenced against 34 module documentation files (`api_reference.md`, `architecture.md`, `data_model.md`, `deployment.md`, and per-module documentation files in the `modules/` subdirectory).

4.2 Analysis Procedure

Each conversation was analyzed using a structured protocol examining six dimensions: operator prompting patterns, model behavior, failure modes, operator compensating behaviors, architectural decision attribution (operator versus model), and quantitative observations including iteration counts, file sizes, and error frequencies. Analysis was conducted by passing each conversation alongside matched documentation to Claude Opus (`claude-opus-4-5`) with a structured research prompt, producing structured Markdown output per session. Results were reviewed and verified against the raw conversation logs by the operator-author.

The operator-author dual role introduces potential observer bias, particularly in the classification of compensating behaviors and architectural decision attribution. No inter-rater reliability measure was computed. Claims derived from qualitative interpretation of operator intent should be read with this limitation in mind. Claims derived from quantitative counts of model outputs (file sizes, iteration counts, turn counts) are not subject to this bias.

4.3 Independent Failure Mode Validation

To address potential circularity in using Claude to analyze Claude conversations, failure mode classification was independently validated using a rule-based classifier operating on keyword patterns and structural heuristics with no LLM involvement. The classifier applied ten to twelve pattern categories per failure mode to detect environmental assumption language

(configuration blindness), interface inconsistency signals (fragmentation), artifact proliferation indicators (accumulation), and unrequested output expansion patterns (scope creep). Applied to the complete corpus of 2,218 operator-model exchanges across all 18 sessions, the classifier identified configuration blindness signals in 8.5% of exchanges (n=189), compensating behavior in 4.0% (n=89), scope creep in 3.1% (n=68), fragmentation in 0.9% (n=21), and accumulation in 0.5% (n=10). 84.7% of exchanges showed no failure mode signal.

To assess whether the 8.7% of failure-mode exchanges represent substantively important events rather than noise, five metrics were applied to the complete corpus. Position analysis found non-uniform clustering: failure mode exchanges concentrated at early turns (architecture establishment) and late turns (deployment and debugging), not uniformly across sessions. Cascade analysis found that 65.5% of failure mode exchanges occurred in clusters of multiple failure modes within five turns, indicating compounding rather than isolated occurrence. Session length correlation was weak (Pearson $r = -0.175$), indicating that the operator contained failure modes without allowing them to dominate session duration. Average resolution cost was 0.4 subsequent turns, indicating rapid model correction once the operator identified and reframed the problem. These metrics characterize the 8.7% failure mode exchanges as episodic, cascading, and position-concentrated rather than uniformly distributed noise.

4.4 Scope and Limitations

This is a single-subject practitioner study (N=1). No findings are statistically generalizable. The operator is also the researcher, introducing observer bias that is partially but not fully mitigated by the rule-based validation. The system was built in a specific domain (Iraqi educational content) with specific constraints (Arabic language, curriculum alignment) that may not represent general-purpose SaaS development. The study measures the development process only: system quality, user satisfaction, and long-term maintainability are not measured. Cost compression figures compare against informal estimates of traditional development costs rather than a controlled baseline. The operator's specific educational background (CS degree with infrastructure-heavy curriculum, prior exposure to systems design) may be a confounding variable that cannot be disentangled from general CS training in this study.

5. FINDINGS

5.1 Corpus Overview

Table 1 summarizes the 18 conversation sessions by turn count and primary focus area.

Session Title	Turns	Primary Modules
AWS infrastructure setup verification	864	deployment, auth, health
Teachers SaaS platform production setup	496	auth, deployment, database
Building new features	524	database, auth, api_endpoints

Content automation system priorities	388	database, analytics, batch
Iraqi Arabic script generation fixes	410	script_generator, callaway
Fixing Flask SaaS application bugs	428	all modules (cross-cutting)
Teachers platform rebuild	371	script_generator, database
AWS dashboard deployment plan	232	deployment, metrics
Fixing errors in uploaded files	271	api_endpoints, database
App troubleshooting and bug fixes	275	script_generator, batch
V11 migration to functional app	196	auth, database, deployment
Subtopic extraction pipeline	86	batch, callaway, data_loader
Customizable essay generator	61	essay_generator, auth
Adding podcast feature	8	script_generator
Self-improving LinkedIn AI system	10	analytics, metrics
Teachers YouTube video generator	7	script_generator, callaway
Kaplen dashboard rebuild	13	analytics, callaway
Removing Arabic characters	6	api_endpoints
TOTAL	3,662	34 modules

Table 1. Conversation sessions by turn count and primary modules.

5.2 Failure Mode 1: Configuration Blindness

The model consistently produced syntactically correct code that failed at runtime because it could not observe the actual state of the deployment environment. This failure mode appeared in 12 of 18 sessions (67%) and accounted for 8.5% of all classified exchanges (n=189) in the rule-based validation.

In the Teachers SaaS platform setup session (496 turns), SSH commands failed with "Permission denied (publickey)" because the model assumed the key file existed at the specified path. In the App troubleshooting session, a 100% failure rate on 944 video transcripts was caused by ANTHROPIC_API_KEY not being set on EC2; the model had no mechanism to detect this without operator diagnosis. In the AWS infrastructure session (864 turns), the model provided CloudFormation YAML fragments without instructions for integrating them into an existing stack, assuming knowledge the operator did not have.

The model treats its execution environment as identical to its specification. It cannot observe network state, file system state, environment variables, or process state. Every configuration-layer failure required the operator to identify the failure class before the model could address it. This maps to the execution failure category in Cemri et al.'s multi-agent failure taxonomy [10], specifically the subcategory of environment-state blindness.

5.3 Failure Mode 2: Fragmentation

The model lost architectural coherence across sessions and within long sessions, producing code that contradicted earlier decisions. This failure mode appeared in 11 of 18 sessions (61%) and accounted for 0.9% of classified exchanges (n=21), concentrated in complex multi-session builds.

In the Fixing errors session (271 turns), 19 bugs were identified across 8 files. Several showed classic fragmentation: function signatures that stopped matching their call sites (register_all_routes missing four parameters that the code inside referenced), database schemas that diverged from code, and duplicate files (long_form_generator.py and long_form_generator_v2.py were identical). In the V11 migration session, versions v12 through v16 each broke a working v11, with the model having no memory of what v11 had established.

This failure mode corresponds directly to what the context engineering literature terms "intent drift" [11] — gradual erosion of alignment with original architectural intent over multi-turn interactions. The emergence of AGENTS.md as an industry standard for persistent agent context confirms that this is a recognized structural limitation rather than a model-specific behavior.

5.4 Failure Mode 3: Accumulation Without Consolidation

The model added system complexity without detecting when complexity had exceeded manageability, and without initiating consolidation without explicit instruction. This failure mode appeared in 9 of 18 sessions (50%) and accounted for 0.5% of classified exchanges (n=10), concentrated in high-output sessions.

The orchestrator file central to the Kaplen build grew from 9KB to 74KB across 81 documented snapshots before the operator intervened. The model never flagged this growth as a problem. When the operator initiated a consolidation day, the file returned to 4KB, with 32% of all iterations on that file being consolidation work, none of it model-initiated.

In the Fixing Flask bugs session (428 turns), the model created 8 documents totalling 75KB and made 7 consecutive "final summary" attempts without completing the repair task. In the Teachers setup session, at least three versioned files were created in a single conversation (app_v8_final.py, app_v9.py, app_v9_updated.py) without consolidation or cleanup.

5.5 Failure Mode 4: Scope Creep

The model expanded output beyond what was requested without operator confirmation. This was the most prevalent failure mode, appearing in 14 of 18 sessions (78%) and accounting for 3.1% of classified exchanges (n=68).

In the Iraqi Arabic fixes session (410 turns), a request to fix script generation produced 10+ documentation files, deployment checklists, and visual summaries alongside the requested code changes. In the Teachers YouTube session, 5 clarifying questions appeared in the first response and 3 in the second, totalling 8 questions before any code was generated, despite the

operator's request being specific and bounded. In the Self-improving AI session, a simple integration question produced 7 complete artifacts: an analysis document, a SQL schema, 4 Python scripts, and a README.

The model's default behavior is to maximize helpfulness by producing more than asked. This is consistent with findings by Chou et al. [6] in their vibe coding study, where practitioners reported that managing over-generation was a recurring challenge. In a structured development context this creates overhead: more files to review, more decisions to validate, more surface area for errors.

5.6 Operator Compensating Behaviors

In response to each failure mode, the operator developed consistent compensating behaviors traceable across multiple sessions. These behaviors were not planned in advance; they emerged as responses to specific failure events and were then reused across subsequent sessions.

Context pre-loading: Because the model has no persistent memory, the operator re-established architectural context at the start of every session. In the Fixing Flask bugs session, the operator's opening prompt was an 18-point structured system description specifying stack, purpose, constraints, and requirements before any diagnosis began. This is the manual equivalent of the AGENTS.md specification pattern [11], which emerged as a formalized standard after the period covered by this study.

Phase-gate prompting: The operator divided complex tasks into explicitly bounded phases and required model confirmation before proceeding. The checkpoint protocol ("say 'next' and I'll move to PART 2") appeared in the AWS infrastructure session and was adopted across subsequent sessions. This directly compensated for scope creep by preventing the model from advancing until the operator validated the current phase.

Clean rebuild decisions: When accumulation exceeded the operator's capacity to manage, a rebuild was initiated rather than continued patching. In the App troubleshooting session, the operator's explicit note states: "Decision made: clean rebuild to eliminate accumulated syntax issues." This required the operator to recognize that the cost of continued patching exceeded the cost of restart — a judgment the model never initiated independently.

Explicit consolidation instructions: The orchestrator's reduction from 74KB to 4KB was operator-instructed. The model executed the consolidation but did not initiate it. This pattern repeated across sessions: the operator observed when system complexity had grown beyond productive bounds and instructed consolidation explicitly.

Decision point forcing: When the model presented multiple options, the operator responded with terse structured answers mapped directly to numbered questions. This eliminated the model's tendency to elaborate on options rather than implement the chosen one.

5.7 Architectural Decision Attribution

Analysis of the conversation corpus suggests a consistent pattern: architectural choices were made by the operator, implementation choices were made by the model. This attribution is based on qualitative review of operator prompts and model responses, subject to the observer bias noted in Section 4.2.

The operator specified: EC2 t3.small, RDS PostgreSQL, S3 for curriculum storage, Nginx as reverse proxy, JWT authentication, Stripe for payments, the Dual-Hash deduplication approach, the sectionalized generation architecture, and the subject-topic-subtopic curriculum hierarchy. None of these decisions appear in model output as spontaneous suggestions that the operator adopted; all appear as operator-specified constraints that the model implemented.

The model contributed: specific Python patterns for each module, SQL schema details within the operator's data model specification, Flask route implementation, and debugging solutions for identified errors. The model produced implementation within operator-defined architectural bounds, not the bounds themselves. This division is consistent with Sarkar's [8] argument that intention specification — not implementation execution — is the critical human contribution in AI-assisted development.

6. QUANTITATIVE SUMMARY

Metric	Value
Total conversations	18
Total operator-model turns	3,662
Longest single session	864 turns
Sessions with 400+ turns	5
Documented modules in final system	34
Total tooling cost	USD 500
Calendar days to production	32
Estimated traditional equivalent cost (informal)	USD 150,000–280,000
Cost compression ratio (vs. informal estimate)	~300x
Total exchanges classified (rule-based)	2,218
Sessions with configuration blindness	12 / 18 (67%)
Sessions with fragmentation	11 / 18 (61%)
Sessions with accumulation failure	9 / 18 (50%)
Sessions with scope creep	14 / 18 (78%)
Configuration blindness (exchange level)	189 / 2,218 (8.5%)
Scope creep (exchange level)	68 / 2,218 (3.1%)
Fragmentation (exchange level)	21 / 2,218 (0.9%)
Accumulation (exchange level)	10 / 2,218 (0.5%)

Failure mode exchanges in cascades	65.5%
Avg resolution cost per failure mode exchange	0.4 turns
Session length / failure density correlation	$r = -0.175$ (weak)
Orchestrator file: initial / peak / post-consolidation	9KB / 74KB / 4KB
Total orchestrator snapshots	81
Consolidation iterations as % of total	32%
Turns to fix bugs estimated at 45-65 min	428
Consecutive summary attempts without task completion	7
Documentation created instead of fixing code (1 session)	75KB
Working versions of 6 built (v11-v16)	1 (v11 only)

Table 2. *Quantitative summary. Cost compression figure is based on informal estimates of traditional development timelines and costs, not a controlled comparison.*

7. DISCUSSION

7.1 On the Separability of Engineering and Coding in This Case

The four failure modes identified here are all failures of the engineering layer that sits above code: maintaining coherent state across sessions, bounding system scope, managing complexity, and diagnosing environment versus implementation failures. None are failures of syntax, library knowledge, or implementation correctness. The model consistently produced syntactically valid code; it consistently failed at the architectural layer.

In this specific case, the operator's engineering competence and coding ability functioned as separable: the operator exercised engineering judgment through prompting, phase management, consolidation decisions, and architectural specification, while delegating all implementation. This is consistent with Sarkar's [8] framing of intention specification as the critical human skill, and with Vulicevic's [7] operator role framework.

Three conditions appear to have enabled this separability in this case. First, the operator's CS background included formal training in operating systems, networking, and systems design — which provided the mental model needed to distinguish configuration failures from implementation failures (configuration blindness) and to recognize version divergence as an architectural problem rather than a random error (fragmentation). Second, the system was built greenfield with no legacy constraints, allowing the operator to specify architecture without needing to read or modify existing code. Third, the operator interacted with a single model through a conversational interface, which meant all system state had to be maintained in natural language — a constraint that forced explicit architectural reasoning that might have been implicit in a traditional development workflow. Whether separability holds without all three conditions is an open question.

Studies of vibe coding by experienced programmers [6, 13] suggest that architectural judgment is exercised differently even within the population of developers, with less experienced practitioners struggling more with the failure modes documented here. This is consistent with the hypothesis that engineering training is a prerequisite for effective operator-directed development, though it does not establish necessity.

7.2 On the Non-Engineer Can Build Narrative

This study does not directly test whether non-engineers can build production systems with AI assistance, because the study contains no non-engineer operators. The finding that one engineering-trained operator successfully did so is a positive demonstration, not a general claim. The rest of this section is inference from the failure mode data, not direct evidence.

The compensating behaviors required to manage the four failure modes draw on knowledge typically acquired through engineering training: recognizing configuration-layer failures, maintaining architectural coherence across sessions, knowing when to rebuild rather than patch. Sarkar and Drosos [13] found that experienced programmers scan AI-generated code through pattern matching informed by architectural intuition. Ge et al. [5] note that the democratization claim in vibe coding literature is empirically underexamined. This study contributes one data point: engineering training was present, was exercised in the compensating behaviors, and appears to have been load-bearing. Whether its absence would have produced a different outcome remains untested.

7.3 Relationship to Existing Failure Mode Taxonomies

The four failure modes identified here have structural parallels in Cemri et al.'s multi-agent failure taxonomy [10]. Configuration blindness maps to execution failures caused by environment-state unawareness. Fragmentation maps to planning failures caused by context loss. Accumulation maps to response failures caused by over-production without self-monitoring. Scope creep maps to planning failures caused by over-specification of unrequested components.

The key difference is context: Cemri et al. studied multi-agent systems with explicit tool use and environment access. The present study examines a single-model conversational interface with no tool access. The similar failure mode patterns across these two very different configurations suggest that some failure modes are properties of LLM behavior in general rather than of specific architectures. This has implications for the design of future human-AI development systems.

7.4 On Failure Mode Frequency and Importance

The session-level distribution — scope creep (78%), configuration blindness (67%), fragmentation (61%), accumulation (50%) — reflects how often each failure mode appeared in at least one exchange per session. The exchange-level counts from the rule-based classifier (8.5%, 3.1%, 0.9%, 0.5%) reflect the proportion of all turns affected. Both measures tell a consistent story: scope creep is the most pervasive, configuration blindness is the most concentrated in

high-stakes deployment sessions, and fragmentation and accumulation are episodic but consequential when they occur.

The cascade finding (65.5% of failure mode exchanges occur in clusters) suggests that failure modes compounded rather than appearing independently. This is consistent with the resolution cost data (0.4 turns average): the operator resolved individual failures quickly, but cascades required sustained intervention. The weak session-length correlation ($r = -0.175$) indicates that the operator's compensating behaviors were effective at containing failure modes without allowing them to dominate session duration.

7.5 Implications for Practice

The compensating behaviors documented here represent an informal methodology for operator-directed LLM development. Each behavior compensates for a specific failure mode. Context pre-loading addresses fragmentation; phase-gate prompting addresses scope creep; explicit consolidation addresses accumulation; operator-level environment diagnosis addresses configuration blindness. The emergence of formal tools addressing the same problems (AGENTS.md for context persistence [11]) suggests these compensating behaviors are not idiosyncratic to this operator but reflect genuine needs in LLM-directed development.

For teams using AI tools alongside existing developers, the failure modes identified here suggest specific points of vigilance: explicit verification of environment configuration rather than assumption; architectural snapshots maintained across sessions; complexity metrics monitored with consolidation scheduled proactively; and output scope bounded by explicit prompting constraints.

7.6 A Practical Operator Protocol

The compensating behaviors, taken together, form a repeatable protocol for operator-directed LLM development. The protocol is distilled from observed practice in this study, not prescribed in advance.

Session initialization: before any implementation request, provide the model with a structured system description covering stack, purpose, active modules, and any constraints established in prior sessions. This is the manual equivalent of AGENTS.md [11] and directly compensates for the model's lack of persistent memory. The initialization prompt should be explicit, not assumed: "Here is the current system state before we begin."

Phase gating: divide each session into explicit phases. Require model acknowledgment before proceeding to the next phase. The gate prevents scope creep by creating a confirmation checkpoint before output expands. The specific protocol used in this build ("say 'next' and I will move to PART 2") is one instantiation; the critical element is the explicit pause for operator validation.

Complexity monitoring: track a proxy metric for system complexity — file sizes, module count, line count, or number of active interfaces. When any proxy metric grows beyond a threshold the operator sets in advance, schedule a consolidation session before adding new features. In this build, the orchestrator file size served as the primary proxy. The 74KB peak before consolidation was recognized retrospectively as too late; a threshold of 30-40KB would have triggered earlier intervention.

Failure mode triage: when a session produces unexpected failures, classify before debugging. The four failure modes map to distinct diagnostic paths: configuration blindness requires environment inspection, not code review; fragmentation requires architectural comparison against a known-good state; accumulation requires consolidation, not new implementation; scope creep requires constraint injection, not elaboration. Applying the wrong diagnostic path wastes turns.

Rebuild threshold: maintain an explicit criterion for when accumulated complexity warrants a clean rebuild rather than continued patching. In this build, the criterion was implicit ("the cost of patching exceeds the cost of restarting"). Making this criterion explicit in advance — for example, "if three consecutive sessions fail to resolve the same class of error, initiate a clean rebuild" — would reduce the decision cost and prevent over-investment in unrecoverable states.

8. CONCLUSION

This study documents the construction of a 34-module production SaaS system by an operator with software engineering training but no direct coding ability, using an LLM as the sole implementation agent across 3,662 documented operator-model turns over 32 days at a cost of USD 500. Analysis of the complete interaction corpus identifies four consistent failure modes (configuration blindness, fragmentation, accumulation, and scope creep) and four corresponding operator compensating behaviors (context pre-loading, phase-gate prompting, explicit consolidation, clean rebuild decisions). Independent validation using a rule-based classifier across 2,218 exchanges confirmed the failure mode distribution without LLM involvement.

In this case, software engineering competence and coding ability functioned as separable skills. The operator's engineering judgment — specifying architecture, recognizing failure classes, bounding scope, managing system complexity — was exercised entirely through natural language without writing code. Whether this separability holds for operators with different training profiles, in different domains, or at different system scales is not addressed by this study and remains an important open question.

These findings sit at the intersection of vibe coding research [5, 6], LLM failure mode analysis [10], and operator-model interaction theory [7, 8]. The primary contribution is not a general claim about what AI can democratize, but a detailed empirical record of one complete

production build — the failure modes that appeared, the behaviors that addressed them, and the specific decisions that distinguished architectural judgment from implementation execution.

ACKNOWLEDGMENTS

Claude (Anthropic) served two roles in this work. First, as the sole implementation agent throughout the Kaplen build described in this study — this role is the subject of the research. Second, as an analytical and drafting assistant: Claude Opus (claude-opus-4-5) was used to analyze conversation logs against documentation and produce structured analytical output per session, and Claude assisted in drafting prose sections of this paper from the author's outlined arguments and data. The independent failure mode validation (Section 4.3) used a rule-based classifier with no LLM involvement to address circularity in the analytical role. All scientific claims, data interpretation, failure mode taxonomy, conclusions, and final editorial decisions are the author's own. The raw conversation corpus, module documentation, and rule-based classifier source code are available as ancillary files.

APPENDIX: RULE-BASED CLASSIFIER PATTERN SPECIFICATION

The rule-based classifier used for independent failure mode validation (Section 4.3) applied the following pattern categories per failure mode. All patterns are Python regular expressions applied case-insensitively to model output text (first 3,000 characters) and operator prompt text (first 2,000 characters). A failure mode is flagged when the combined hit count across model and operator patterns meets or exceeds the threshold specified per mode.

Configuration Blindness (threshold: 2 combined hits):

Model patterns: environment variable or getenv references; permission denied or access denied errors; no such file or path does not exist; port already in use; connection refused or timed out; SSH or PEM key references; missing or undefined variable or credential; "make sure you have" or "ensure that" constructions; service not running; database not found. Operator patterns: error or failed or not working; still getting the same error.

Fragmentation (threshold: 2 combined hits):

Model patterns: does not match or mismatch or inconsistent; versioned filename patterns (`_v2`, `_final`, `_updated` with `.py/.js/.sql` extension); undefined function or variable or import; schema mismatch or column not found; wrong function or parameter or signature. Operator patterns: still broken or same error; it used to work or worked previously.

Accumulation Without Consolidation (threshold: 2 combined hits):

Model patterns: creating a new or another file/function/module; "here's an updated/new/revised version"; in addition or additionally; summary or documentation or guide or checklist followed by colon; "let me also add or create"; multiple new file mentions (4 or more `.py/.js/.sql/.md` files referenced). Operator patterns: just fix or change this; consolidate or clean up or simplify or refactor.

Scope Creep (threshold: 2 combined hits):

Model patterns: "I've also added/included/created"; bonus or extra or additionally; "I also added/created/wrote"; "you might also want/consider"; "I took the liberty" or "I went ahead and"; complete rewrite or overhaul; multiple code blocks (4 or more triple-backtick pairs). Structural pattern: operator prompt under 30 words with model response over 500 words.

Compensating Behavior (threshold: 1 hit in operator patterns only):

Operator patterns: "here's the context/background/architecture/system"; "before you/we start/begin/continue"; step N or phase N or part N references; "say next/continue/proceed when"; consolidate or clean up or start fresh or rebuild; only do/change/fix this or one thing; do not add/change/create anything else; wait for my next instruction.

The full classifier source code is provided as ancillary file `classify_eval.py`. The importance assessment scripts (position clustering, cascade detection, operator response rate, session correlation, resolution cost) are provided as ancillary file `assess_importance.py`. Both files are self-contained Python scripts with no external LLM dependencies.

REFERENCES

- [1] C. Antonio et al., "The Impact of Large Language Models (LLMs) on Code Review Process," arXiv:2508.11034, Aug. 2025.
- [2] GitClear, "Coding on Copilot: 2023 Data Suggests Copilot Leads to More Bugs," GitClear Research, 2024.
- [3] J. Becker, N. Rush, E. A. Barnes, and D. B. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," arXiv:2507.09089, July 2025.
- [4] A. Karpathy, "Vibe coding," X (formerly Twitter), Feb. 2025. [Online]. Available: <https://x.com/karpathy/status/1886192184808149393>
- [5] Y. Ge et al., "A Survey of Vibe Coding with Large Language Models," arXiv:2510.12399, Oct. 2025.
- [6] Y.-H. Chou et al., "Building Software by Rolling the Dice: A Qualitative Study of Vibe Coding," arXiv:2512.22418, Dec. 2025.
- [7] Z. Vulicevic, "The AI Operator: A Reference Role for Human-AI Systems," Zenodo, Dec. 2025, doi: 10.5281/zenodo.18105972.
- [8] A. Sarkar, "Intention Is All You Need," arXiv:2410.18851, Oct. 2024.
- [9] S. Daniotti et al., "Who is using AI to code? Global diffusion and impact of generative AI," Science, vol. 391, no. 6787, pp. 831-835, Jan. 2026.
- [10] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" arXiv:2503.13657, Mar. 2025.
- [11] Augment Code, "Context Engineering: Enhancing Agentic Swarm Coding through Intent, Environment, and System Memory," [augmentcode.com](https://www.augmentcode.com), Sept. 2025. [Online]. Available: <https://www.augmentcode.com/guides/context-engineering>
- [12] L. Barke et al., "Should I Give Up Now? Investigating LLM Pitfalls in Software Engineering," ACM TOSEM, 2026, doi: 10.1145/3801972.

- [13] A. Sarkar and I. Drosos, "Vibe coding: programming through conversation with artificial intelligence," arXiv:2506.23253, 2025.
- [14] E. Mendes et al., "The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review and Mapping Study," arXiv:2507.03156, 2026.
- [15] B.-C. Chen et al., "Unleashing the Potential of Prompt Engineering for Large Language Models," arXiv:2310.14735, Oct. 2023.
- [16] S. Schulhoff et al., "The Prompt Report: A Systematic Survey of Prompting Techniques," arXiv:2406.06608, June 2024.